Skim Before You Reason: Half the Cost, Similar Accuracy in Agentic Search

Quyet V. Do
Virginia Tech, VA 24061, USA
quyetdo@vt.edu

Abstract

Deep search agents answer complex information-intensive questions by iteratively issuing queries to a search engine, reading the retrieved documents, and reasoning over the accumulated evidence. A common search agent design injects all retrieved documents directly into the model's context. As the search trajectory grows, this context accumulates redundant and irrelevant content, which increases inference cost and dilutes the evidence that actually matters. Inspired by the common *skim-then-read* practice, we introduce *Retrieve-from-the-retrieved* (RR), a **low-cost** add-on module that condenses retrieved documents *before* they enter the agent's context: it first applies sparse-search retrieval over chunks of retrieved documents, then applies dense-search retrieval over the surviving chunks, finally passing the remaining chunks to the agent for reasoning. Averaged over four information-seeking benchmarks, RR regularly cuts the average inference cost of REACT by 30% to 50% while matching or improving its accuracy. Analyses further show that the effect holds across backbone models of different sizes, families, and thinking budgets, and that RR can help reduce reasoning errors in large contexts while mitigating evidence loss due to post-retrieval processing. Our code is available at https://github.com/vtpss/retrieve-from-the-retrieved.

1 Introduction

Search is a fundamental capability for answering complex and information-intensive questions, enabling systems to access up-to-date knowledge, discover diverse sources, and ground their reasoning in external evidence. Recent advances in LLMs have elevated search from a single-shot retrieval operation to agentic search, in which an LLM iteratively formulates queries, retrieves relevant information, reasons over the evidence, and uses intermediate findings to guide subsequent searches (Vu et al., 2024; Li et al., 2025; Jin et al., 2025; Team et al., 2025; Wei et al., 2025; Gupta et al., 2026; Chen et al., 2026c). Such long-horizon search agents have demonstrated impressive capabilities on challenging tasks that require multi-step exploration and synthesis across heterogeneous sources, and have become an important foundation for applications such as deep research (Google, 2025; OpenAI, 2025).

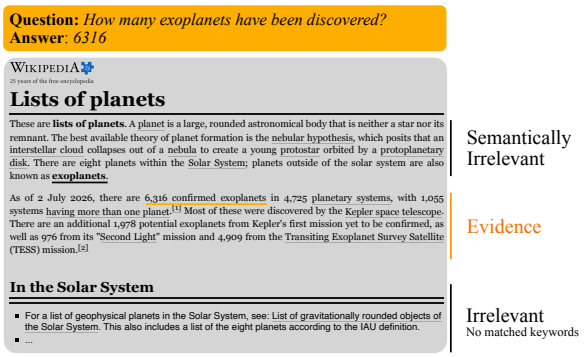


Figure 1: **Motivation**. We aim to reduce the number of irrelevant passages that enter search agents' LLM contexts. Given the query *"How many exoplanets have been discovered?"* and a resulting web page, we observe that most paragraphs on the page are irrelevant to the question. As illustrated by the third paragraph, many can be filtered out through lexical matching (resembling *skimming*). The remaining paragraphs may still be irrelevant, so vector search (resembling *reading*) can further filter out semantically irrelevant passages.

Despite this progress, *context optimization in agentic search for both efficiency and effectiveness remains a challenge*. As the search trajectory grows, it introduces substantial amounts of redundant or irrelevant retrieval content, increasing context length and inference cost while potentially distracting the model from the evidence that actually matters. Existing efforts largely operate at certain levels of the system: retrieval-level methods optimize the retriever, indexing strategy, or retrieval unit (Chen et al., 2026b,a; Meng et al., 2026), while

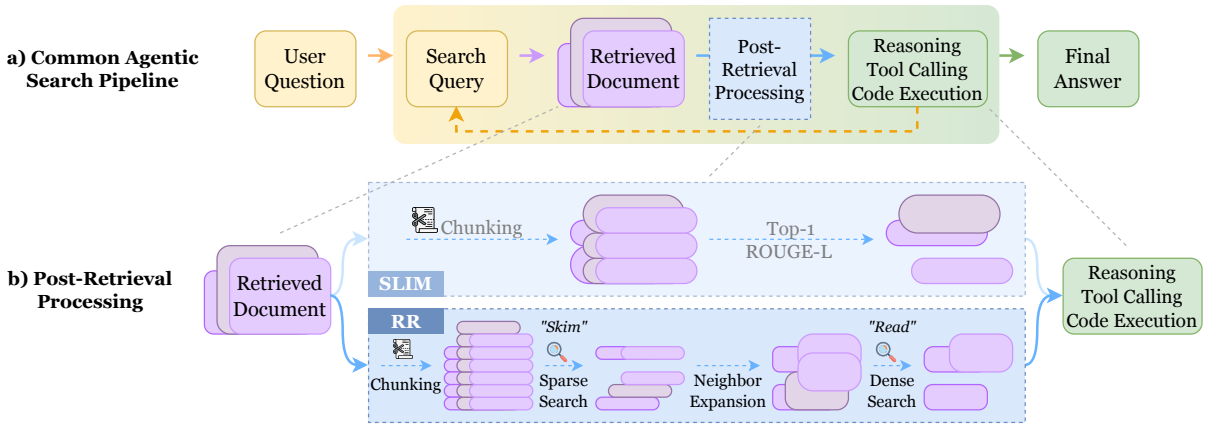


Figure 2: **A common agentic search pipeline and our post-retrieval processing method RR**. Compared with SLIM, a method that returns the most lexically relevant section from each retrieved document, RR uses smaller chunks and treats all retrieved documents as a searchable collection. This design helps filter irrelevant content while retaining evidence for different reasoning patterns.

context-level methods summarize or manipulate the accumulated interaction history (Wu et al., 2026; Ye et al., 2026; Liu et al., 2026) or treat the context as an external environment for recursive processing (Zhang et al., 2026a). These approaches leave an important dimension largely underexplored:

> **RQ:** *Given a set of documents newly retrieved by a search engine, how can we efficiently identify the most relevant chunks to include in the LLM's context?*

This question is particularly important in realistic settings such as web search, where the search engine is a remote black box and its retrieval pipeline cannot be modified. Recently, Yen et al. (2025) proposed a post-retrieval processing method, SLIM, that lets only the most lexically relevant chunk from each retrieved document enter the LLM's context. However, our later analysis shows that this approach is prone to evidence loss.

We approach this problem by drawing inspiration from human information-seeking behavior. Pernice et al. (2014) found that, in online information-seeking tasks, people do not commit to reading all web content at the outset. Instead, they first skim through a document and then locate and read important information. Inspired by this study, we design a module called *Retrieve-from-the-retrieved* (RR), illustrated in Figure 2. It first filters retrieved documents for the most lexically relevant information using sparse search over document chunks (resembling *skimming*), then filters the remaining chunks for semantic relevance using dense search (resembling *reading*). The resulting chunks are passed to the agent for reasoning. The efficiency and effectiveness of RR stem from two design choices: reducing the number of input tokens to the backbone LLM, and using sparse and dense search for context filtering. This approach consumes fewer resources than LLM-based compaction while remaining flexible enough to preserve evidence across different retrieval-reasoning patterns.

We evaluate RR on four short-answer information-seeking benchmarks of increasing search complexity—LiveNewsBench (Zhang et al., 2026b), Seal-Hard (Pham et al., 2026), DeepSearchQA (Gupta et al., 2026), and BrowseComp-Plus (Chen et al., 2026c)—by integrating it into two agentic frameworks, REACT (Yao et al., 2023) and RLM (Zhang et al., 2026a). Averaged over the benchmarks and across two backbone LLMs, RR generally lowers cost while matching or improving accuracy: With GEMMA-4-E4B, REACT paired with RR (REACT+RR) attains 38.76 at $3.47 per thousand samples versus 35.11 at $7.36 per thousand samples for REACT (53% lower cost), and with GPT-OSS-120B it attains 47.61 at $7.01 versus 47.51 at $24.22 for REACT (71% lower cost). The effect is smaller, but generally remains visible, for RLM. Further analyses show that RR also helps to reduce reasoning errors of REACT over long contexts. More importantly, RR is more effective in preserving evidence than SLIM, thus more effective in preserving the task accuracy of REACT across benchmarks. The cost reduction at comparable accuracy also holds on

LiveNewsBench and Seal-Hard across backbone models of different sizes, families, and thinking budgets, making REACT generally 30% to 50% cheaper while maintaining or improving accuracy.

Our contributions are as follows: **First**, we study an underexplored dimension of search agents: post-retrieval optimization for efficiency and performance. **Second**, we propose a simple, low-cost, yet effective post-retrieval optimization module for search agent frameworks to reduce irrelevant context. **Third**, we empirically validate RR across four benchmarks, two agent frameworks, and a range of backbone LLMs, showing regular 30% to 50% cost reduction at matched or improved accuracy, without a systematic loss in retrieval recall.

2 Methodology

We will first briefly describe a common design of agentic search, then introduce our post-retrieval processing method RR.

2.1 Common Agentic Search Pipeline

As illustrated in Figure 2, agentic search lets an agent iteratively search for information and reason over the retrieved information to answer a user's question. In general, a search iteration consists of three main steps: 1) **Query Formulation:** determine the next search query given the original user question, retrieved information, and the reasoning process so far; 2) **Search-and-Retrieve:** search the available index with the query and return matched documents, such as web content in web search or local documents from a local index; and 3) **Reasoning:** use all available information, including the newly retrieved documents and reasoning history, to determine the next actions, propose candidate answers, manipulate the working context, request another search, or generate the next search query.

This formulation covers both popular REACT and RLM frameworks. While REACT explicitly scaffolds the aforementioned iteration structure, RLM implicitly follows a similar structure as it generates code to search with a query and retrieve results, then reason over the context.

2.2 RR

Our Retrieve-from-the-retrieved (RR) is a post-retrieval algorithm that processes the retrieved documents before they are injected into the context, assuming the retriever is a black box (e.g., a commercial search engine) and the algorithm has access to the current search query. **Unlike previous related approaches, RR views the retrieved documents as another database to retrieve from**. RR consists of the following components: chunking, sparse-search filtering (accompanied by neighbor aggregation), and dense-search filtering. The overall pipeline is illustrated in Figure 2.

Chunking Our chunking approach aims to produce clean chunks: each chunk should be a semantically complete information block, and chunks should be neither too long (to avoid a high proportion of irrelevant context) nor too short (to avoid context fragmentation or information loss). We use the following rules to chunk *a retrieved document*: 1) In the search-and-retrieve step, retrieved web pages are stored in Markdown format. If the database is offline (e.g., in case of BrowseComp-Plus), we keep the retrieved documents as is. 2) When available, the Markdown heading is used to split the document into sections. After that, sections are further split into paragraphs, which serve as chunks. If a paragraph has more than 256 tokens, we split it by sentence into multiple chunks of roughly 128 tokens.[1] 3) A Markdown table/code block/figure block, regardless of size and number of tokens, is treated as a chunk.

Sparse-search Filtering (`Sparse`) and Neighborhood Expansion (`N.E.`) From the chunks resulting from the chunking step, we first build a BM25+ index (Lv and Zhai, 2011). Then, we retrieve the top k_1 chunks based on their relevance to the query in the current iteration. This step allows us to quickly eliminate irrelevant chunks based on vocabulary, resembling the scanning step in human information-seeking behavior. Following common practice in information retrieval (Wang et al., 2024; Guo et al., 2025b), we also perform neighborhood expansion after sparse-search filtering. For each chunk retrieved in this step, we include its previous and next chunks (if they exist) to capture related information, entities, and relationships, thereby improving evidence recall. This is important because a chunk may not be self-contained and may require its neighboring chunks to be fully understood, or evidence may be missed by lexical matching. These two steps together serve as a negligible-cost context optimization step before the reasoning step with the backbone LLM.

Dense-search Filtering (`Dense`) With k_1 chunks from the previous step, we then build a vector index

[1] With respect to the `o200k_base` encoding

using a dense embedding model. We retrieve the top k_2 chunks based on their semantic relevance to the query in the current iteration. This step allows us to further eliminate irrelevant chunks based on semantic meaning, serving as a low-cost context optimization step before the reasoning step with the backbone LLM.

Default Configuration In notation, our RR module can be represented as: RR = `Sparse+N.E.+Dense`. It uses the BM25+ implementation in `bm25s` (Lù, 2024) with default hyperparameters to build the sparse indexes, and uses `pylate` (Chaffin and Sourty, 2025) with the 149M-parameter `lightonai/Reason-ModernColBERT` (Chaffin, 2025) as the embedding model to construct late-interaction dense indexes. The default values for k_1 and k_2 are 40 and 15, which we refer to as `medium` *retrieval effort*.

3 Experiments

3.1 Benchmarks and Metrics

We evaluate our approach on four short-answer information-seeking benchmarks that span different levels of search complexity.

- **LiveNewsBench** (Zhang et al., 2026b): It features fresh question-answer pairs from recent news articles. The questions are intentionally difficult, requiring multi-hop search queries, page visits, and reasoning, making it well-suited for evaluating agentic search behavior. We use the January 2026 release, the latest available at the time of our experiments, which contains 200 samples.
- **Seal-Hard** (Pham et al., 2026): A set of 254 fact-seeking questions for which web search tends to return conflicting, noisy, or unhelpful results. It is used to assess factual accuracy and reasoning capabilities of search agents.
- **DeepSearchQA** (Gupta et al., 2026) and **BrowseComp-Plus** (Chen et al., 2026c): The two benchmarks consist of difficult multi-step information-seeking tasks across fields, designed to evaluate an agent's ability to execute complex search plans. For each benchmark, we randomly sample 200 questions to form the test set for experiments.

By default, we perform web search with DuckDuckGo via the `ddgs` package,[2] which returns the top $k = 10$ URLs, and crawl the retrieved pages into Markdown with `trafilatura` (Barbaresi, 2021). For BrowseComp-Plus, we use the BM25 index provided with the corpus as the search engine. We use each benchmark's provided LLM grader (with backbone GPT-OSS-20B) to assign answer accuracy.

[2] https://github.com/deedy5/ddgs

3.2 Baselines

We integrate RR into the search-and-retrieve stack under two agentic frameworks, REACT (Yao et al., 2023) and RLM (Zhang et al., 2026a), yielding REACT+RR and RLM+RR, and compare against the following baselines:

- **Zeroshot**: the backbone LLM answers directly from internal knowledge, without searching. It measures the model's prior knowledge of the benchmark questions.
- **Minimal**: the backbone LLM answers after a single search. It measures the effectiveness of a single retrieval step, or equivalently the retrieval complexity of the question: if this baseline performs well, an iterative agent is unnecessary.
- **RLM**: the Recursive Language Model treats the context, including retrieved content, as an external environment that it decomposes and processes recursively. We expose the default search-and-retrieve stack to RLM as a tool.
- **REACT**: the standard framework that interleaves reasoning and search. Its context is automatically compacted by the backbone LLM once it reaches 66% of the context window.
- **SLIM** (Yen et al., 2025): a REACT variant that, for each retrieved document, keeps the chunk with the highest lexical similarity to the query. We follow its default implementation setting.

We run every baseline **once** with each of the two backbone LLMs, GEMMA-4-E4B (Gemma-Team et al., 2026) and GPT-OSS-120B (OpenAI et al., 2025), with temperature $T = 0.0$. We deploy GEMMA-4-E4B (`fp8` quantization) on an NVIDIA A30 GPU (24G VRAM) and GPT-OSS-120B on an NVIDIA A100 GPU (80G VRAM), both using vLLM (Kwon et al., 2023). We use each model's default (i.e., recommended) hyperparameters to maximize its performance.

3.3 Results

Table 1 reports average accuracy and estimated cost in US dollars per thousand samples; the cost model is given in subsection A.4. We report cost rather than token counts, as changing the search-

	LiveNewsBench	Seal-Hard	DeepSearchQA	BrowseComp-Plus	Avg.
GEMMA-4-E4B					
Zeroshot	1.5	5.51	6.0	0.5	3.38
Minimal	1.5 ($0.38)	27.64 ($1.13)	4.0 ($0.58)	0.0 ($2.01)	8.29 ($1.03)
RLM	**56.5** ($4.47)	25.98 ($6.37)	11.5 ($10.55)	**1.5** ($21.38)	23.87 ($10.69)
RLM+RR	55.0 ($3.88)	**28.35** ($4.50)	**14.0** ($8.54)	0.5 ($13.35)	**24.46** ($7.57)
REACT	71.5 ($2.52)	31.43 ($3.15)	19.5 ($10.98)	**18.0** ($12.78)	35.11 ($7.36)
SLIM	72.0 ($1.64)	27.17 ($2.18)	19.5 ($5.72)	16.0 ($2.55)	33.67 ($3.02)
REACT+RR	**74.5** ($1.81)	**37.55** ($2.32)	**25.5** ($6.87)	17.5 ($2.86)	**38.76** ($3.47)
GPT-OSS-120B					
Zeroshot	4.5	18.50	18.0	1.5	10.63
Minimal	5.5 ($0.51)	30.16 ($2.14)	20.5 ($0.99)	2.0 ($3.53)	14.54 ($1.79)
RLM	38.0 ($12.99)	24.51 ($12.08)	16.0 ($21.52)	**1.5** ($28.53)	20.00 ($18.78)
RLM+RR	**39.0** ($11.42)	**28.74** ($12.31)	**22.0** ($21.43)	1.0 ($24.68)	**22.69** ($17.46)
REACT	68.5 ($3.02)	39.04 ($4.79)	**40.0** ($14.19)	**42.5** ($74.88)	47.51 ($24.22)
SLIM	66.5 ($2.33)	34.65 ($2.77)	33.5 ($7.70)	36.0 ($11.75)	42.66 ($6.14)
REACT+RR	**69.0** ($2.16)	**43.43** ($2.76)	38.0 ($9.73)	40.0 ($13.41)	**47.61** ($7.01)

Table 1: **Main experiments.** The table reports average **accuracy** and **estimated cost per thousand samples (in USD)** for each baseline and benchmark, except for *Zeroshot*. **Bold** numbers indicate the better result between each base framework and its RR variant. RR generally maintains or improves accuracy while reducing the base framework's cost by 30%-50%.

and-retrieve stack may affect multiple components of the framework, thus affecting the overall cost.

Averaged over the four benchmarks, RR reduces cost for both frameworks while matching or improving accuracy. With GEMMA-4-E4B, REACT+RR attains 38.76 at $3.47 versus 35.11 at $7.36 for REACT (+3.65 points at 53% lower cost), and RLM+RR attains 24.46 at $7.57 versus 23.87 at $10.69 for RLM. With GPT-OSS-120B, REACT+RR attains 47.61 at $7.01 versus 47.51 at $24.22 for REACT (+0.10 points at 71% lower cost), and RLM+RR attains 22.69 at $17.46 versus 20.00 at $18.78 for RLM.

We note that *the cost reductions and performance gains brought by* RR *are not always consistent or statistically significant*[3] across all benchmarks, base frameworks, and backbone models. For example, the cost reduction is significant for all benchmarks and backbones with REACT, but not for RLM with GPT-OSS-120B on Seal-Hard and DeepSearchQA. Similarly, the accuracy gains are significant on LiveNewsBench and Seal-Hard for REACT with both backbones. On the other hand, on DeepSearchQA, REACT+RR with backbone GPT-OSS-120B is slightly worse than REACT. On BrowseComp-Plus, the base framework has 0.5 and 2.5 points higher accuracy than its combination with RR.

Comparing REACT+RR with SLIM examines whether retrieving multiple relevant chunks across documents is more effective than retaining one large chunk per document. REACT+RR is slightly more expensive than SLIM but significantly more accurate on average: 38.76 versus 33.67 with GEMMA-4-E4B (+5.09 points, $3.47 versus $3.02) and 47.61 versus 42.66 with GPT-OSS-120B (+4.95 points, $7.01 versus $6.14).

Meanwhile, both Zeroshot (3.38 and 10.63 average for the two backbones) and Minimal (8.29 and 14.54) trail the iterative agents by a wide margin, indicating that these questions require iterative search of fresh information rather than a single retrieval step or pretraining knowledge.

Last but not least, latency measurements show a significant latency reduction on LiveNewsBench thanks to RR for both backbones (one-sided Welch t-tests, $\alpha = 0.05$), while the effect varies across the other measured benchmarks (see subsection A.5).

3.4 Ablation Study

We ablate the components of RR to assess their individual contributions. We hold the framework and backbone fixed to REACT+RR and GEMMA-4-E4B, and evaluate on LiveNewsBench, Seal-Hard, and DeepSearchQA, which represent different levels of

[3]According to one-sided t-tests with $\alpha = 0.05$.

Variant of RR	LiveNewsBench	Seal-Hard	DeepSearchQA	Avg.
Default configuration	74.5 ($1.81)	37.55 ($2.32)	25.5 ($6.87)	**45.85** ($3.67)
`Sparse+N.E.`	**75.5** ($1.69)	35.08 ($2.11)	25.0 ($6.98)	45.19 ($3.59)
`Dense`	66.5 ($1.71)	**39.84** ($1.78)	**26.0** ($5.74)	44.11 ($3.08)
`Sparse`	68.0 ($1.57)	30.95 ($1.86)	18.5 ($5.72)	39.15 ($3.05)
`Sparse+Dense`	63.5 ($1.65)	33.47 ($1.82)	21.0 ($5.55)	39.32 ($3.01)
`Low` retrieval effort	67.5 ($1.67)	36.76 ($1.65)	20.0 ($5.55)	41.42 ($2.96)
`High` retrieval effort	**75.5** ($1.80)	37.05 ($2.13)	24.0 ($7.99)	45.52 ($3.98)
Fix-length chunking	73.5 ($1.97)	29.25 ($2.62)	20.5 ($7.38)	41.08 ($3.99)

Table 2: **Ablation studies with REACT and GEMMA-4-E4B**. We compare variants of RR with different filtering setups, retrieval efforts, and chunking strategies. The table reports accuracy and estimated cost (in USD) per thousand samples for each variant and benchmark. **Bold** numbers indicate the best performance on each benchmark across all variants. While one variant may prevail on some benchmarks, the default configuration consistently achieves strong performance across all benchmarks and the best average performance.

search complexity. Table 2 reports the results.

Filtering. We explore four alternatives to the default filtering configuration: `Sparse+N.E.`, `Dense`, `Sparse`, `Sparse+Dense`.

- `Sparse+N.E.` removes dense-search filtering and injects the retrieved content after the neighbor expansion step into the search agent's context. It leaves average accuracy and cost essentially unchanged (45.19/$3.59 vs 45.85/$3.67), while making the workflow simpler.
- `Dense` skips sparse-search filtering and neighbor expansion, leaving only dense-search filtering. That means all chunks of the original retrieved documents are encoded and ranked by the dense retriever. In theory, this filtering increases the recall of evidence, thus potentially improving accuracy. This is partially reflected in the experimental results, which show that it improves accuracy on Seal-Hard (39.84 versus 37.55 for the default) and DeepSearchQA (26.0 versus 25.5), but lowers accuracy on LiveNewsBench (66.5 versus 74.5). It suggests that a dense-only filtering configuration is a promising alternative on more complex benchmarks, but it is not stable across benchmarks.
- `Sparse` removes dense-search filtering and neighbor expansion, leaving only sparse-search filtering. Despite the cost reduction, due to the lower recall of sparse search, it significantly lowers accuracy on all three benchmarks, from 45.85 to 39.15 on average. This indicates that aggregating adjacent text spans is important for sparse filtering, as it avoids fragmenting evidence across chunk boundaries.
- Finally, `Sparse+Dense` removes neighbor expansion between sparse and dense search filtering. Similar to `Sparse`, it suffers from lower recall of evidence than the default filtering, thus lowers accuracy on all three benchmarks.

Retrieval effort. Lowering the retrieval effort (from the default `medium` ($k_1 = 40, k_2 = 15$) to `low` ($k_1 = 30, k_2 = 10$)) reduces average accuracy from 45.85 to 41.42 while also lowering cost ($3.67 to $2.96). Raising it to `high` ($k_1 = 50, k_2 = 20$) yields an average accuracy of 45.52 at $3.98, i.e., no clear gain over the default despite a slightly higher cost. That justifies our default choice of $k_1 = 40, k_2 = 15$ as a reasonable default hyperparameter setting. We also observe that higher retrieval effort doesn't always mean larger cost, as higher retrieval effort may provide enough evidence with fewer search calls, which can reduce the overall cost.

Chunking. Replacing document-structure chunking with naive fixed-length chunking lowers accuracy and slightly increases cost on all three benchmarks: 73.5/$1.97, 29.25/$2.62, and 20.5/$7.38 versus the defaults of 74.5/$1.81, 37.55/$2.32, and 25.5/$6.87. This suggests that when cues about document structure are available, structure-based chunking can make the search system more effective and efficient.

Takeaway: Overall, while one variant may prevail on some benchmarks, the default configuration consistently achieves strong performance across all benchmarks and the best average performance.

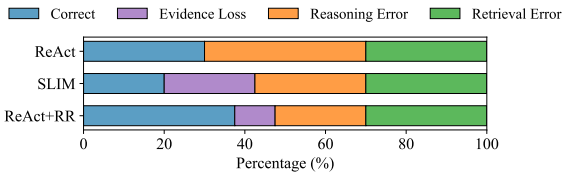


Figure 3: **Failure modes** of REACT variants with the GEMMA-4-E4B backbone on Seal-Hard. Compared with SLIM, REACT+RR reduces failures caused by evidence loss during post-retrieval processing.

3.5 Analysis

Failure modes. We analyze the failure modes of REACT+RR, REACT, and SLIM on Seal-Hard. We categorize the failures into three types:

- *Retrieval Error*: The retrieved content does not contain enough evidence, and thus the backbone LLM cannot answer correctly. This error can occur when questions are designed to be adversarial to search engines or when the search agent commits to an answer too early.
- *Reasoning Error*: The retrieved content contains enough evidence, but the backbone LLM fails to reason correctly. The error can be due to the reasoning complexity of the question w.r.t. the backbone LLM.
- *Evidence Loss*: The retrieved content contains enough evidence, but the post-retrieval processing, namely SLIM and RR, accidentally removes some evidence pieces from the context, and thus the backbone LLM cannot answer correctly.

We randomly select 40 questions from Seal-Hard, manually analyze the failure modes, and report the share of each failure mode in Figure 3. Compared with SLIM, REACT+RR reduces the share of failures due to evidence loss from 22.5% to 10%. This suggests that RR is more effective than SLIM at retaining evidence. Furthermore, the share of reasoning errors decreases from 40% to 22.5% when applying RR to REACT, indicating that cleaner context provided by RR can facilitate better reasoning. We note that retrieval errors persist across the original REACT and both its variants, which is reasonable because neither SLIM nor RR directly optimizes retrieval recall.

Takeaway: RR is more effective than SLIM in retaining evidence, and cleaner context brought by RR can facilitate better reasoning.

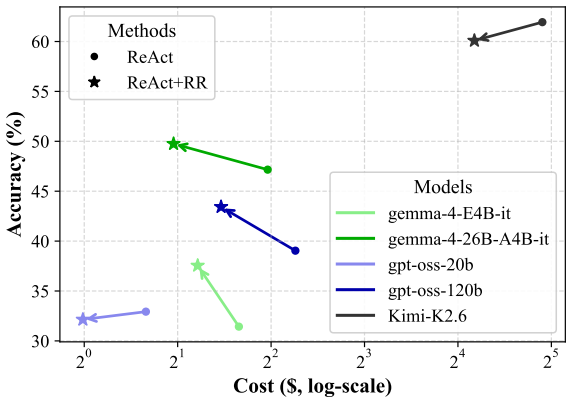


Figure 4: **Performance and cost on Seal-Hard across backbone models**. As in the main experiment, RR generally reduces the cost of REACT by 30%-50% while maintaining or improving accuracy across backbone models of different sizes, families, and thinking budgets.

Backbone model size and type. We study the effect of RR with a variety of backbone LLMs, as larger models may be more robust to context noise and therefore benefit less from our method. Our choices of backbone LLMs include the GEMMA-4-E4B/GEMMA-4-26B-A4B (non-thinking) and GPT-OSS-20B/GPT-OSS-120B model pairs, as well as the trillion-parameter model KIMI-K2.6. In Figure 4, we compare REACT+RR against REACT on two benchmarks, LiveNewsBench and Seal-Hard. The pattern from the main experiment recurs: except for the KIMI-K2.6–LiveNewsBench pair, on which RR has a negligible effect, our method regularly reduces REACT's cost by 30% to 50% while maintaining or improving performance.

Takeaway: RR is effective across backbones of different sizes, families, and thinking budgets with a regular 30% to 50% cost reduction at on-par or better accuracy.

4 Related Work

Framework-Level Context Optimization for Agentic Search Agentic search involves a loop of reasoning and search, where the agent iteratively formulates queries, retrieves documents, and updates its candidate answers (Vu et al., 2024; Li et al., 2025; Jin et al., 2025; Team et al., 2025; Wei et al., 2025; Gupta et al., 2026; Chen et al., 2026c). This loop accumulates verbose search content in the agent's context, leading to possible con-

text window limitations, increased computational costs, and task performance degradation. To tackle these challenges, xRAG (Cheng et al., 2024) and ACC-RAG (Guo et al., 2025a) deeply integrate retrieval content into the backbone LLM's *representation space*, thus avoiding the need to inject verbose retrieval content into the context. At the *prompt level*, REACT-style frameworks like ReSum (Wu et al., 2026), AgentFold (Ye et al., 2026), and PenSIEVE (Liu et al., 2026) focus on summarizing or manipulating the *whole interaction history* to reduce context size. These methods aim to improve efficiency without compromising performance. On the other hand, Recursive Language Models (RLM) (Zhang et al., 2026a) treat the *whole context*, including retrieval content, as an external environment that the model programmatically decomposes and recursively processes. Unlike these methods, our work studies a fundamentally different dimension: the freshly retrieved documents which are not yet injected into the context.

Retrieval-Level Optimization Retrieval quality directly affects both the efficiency and effectiveness of search agents. Many aspects of retrieval can be optimized, including retriever models and algorithms, retrieval units, and rerankers (Wang et al., 2024; Meng et al., 2026). For example, AgentIR (Chen et al., 2026b) and ITER (Chen et al., 2026a) train embedding models that jointly encode the agent's reasoning trace and query to improve retrieval quality. Meng et al. (2026) show that passage-level indexes can be more efficient than full-document indexes for deep search, while SIEVE (Wang et al., 2026) defines a Boolean query language over webpage fields such as titles and headings, so that the agent can filter candidate pages and fetch only selected sections. Beyond common semantic index designs, DCI (Li et al., 2026; Subramanian et al., 2025) and RISE (Zhuang et al., 2026) explore direct agent interaction with raw files through bash commands. These methods assume control of, or structured access to, the retrieval stack—an assumption that breaks when the search engine is a remote black box. Our work does not require control of the retrieval stack; instead, it focuses on optimizing retrieved documents before they are injected into the context. Relevant existing techniques include Reason-in-Documents (Li et al., 2025), which *relies on LLMs* to condense retrieval content, and SLIM (Yen et al., 2025), which uses a ROUGE-L similarity metric to identify *the most relevant chunk of each document*. Our method, RR, shares the idea of inexpensive filtering with SLIM, while aiming to retain more useful context by collecting the most *lexically and semantically* relevant chunks *across all retrieved documents*. In general, RR views the retrieved documents as another database to retrieve from.

5 Conclusion

We introduced RR, a low-cost post-retrieval processing method that filters retrieved documents with sparse and dense search, so that irrelevant parts of the retrieved documents do not enter and dilute the search agent's context. When added to REACT and RLM, RR reduces average inference cost by 53% to 71% across four information-seeking benchmarks while matching or improving average accuracy, without systematically degrading retrieval recall. The gains hold across backbone LLMs and question types, indicating the robustness of our approach. RR is thus a simple, orthogonal lever on the efficiency of deep search. Extending it to information-seeking tasks with long-form answers and to locally hosted search stacks is left to future work.

Limitations

Though our method is applicable to general search problems, our experiments mainly focus on web search. In local search, index design, including preprocessing mechanisms and choices of retrieval unit, directly affects the contribution of RR; evaluating different local index designs is therefore outside the scope of this work. Our experiments also focus on short-answer information-seeking tasks, leaving long-form answer generation to future work. In addition, we do not compare RR with other strong baselines such as ReSum and AgentFold. ReSum is similar to REACT in our experiments, whereas RR and AgentFold pursue fundamentally different context-optimization goals, making a direct comparison less informative; we leave a broader comparison to future work. Finally, we do not explore alternative sparse search methods, embedding or reranking models, or search backends, which we leave for future work.

Acknowledgments

We thank Tu Vu for helpful discussions and feedback on the paper.

AI Use Statement

AI assistants (e.g., ChatGPT, Claude Code) are used in language polishing, LaTeX formatting, and code debugging. All research concepts, experimental methods, data analyses, findings, and conclusions were independently developed and validated by the authors.

Ethical Considerations

Our experiments are conducted on publicly available datasets, models, and tools. Our work inherits the ethical considerations of the underlying datasets and models, including potential biases in the data and limitations of the models. Nonetheless, our work focuses on improving the efficiency of search agents rather than expanding their capabilities, and we do not foresee any direct ethical concerns arising from our work.

References

Adrien Barbaresi. 2021. Trafilatura: A Web Scraping Library and Command-Line Tool for Text Discovery and Extraction. In *Proceedings of the Joint Conference of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, pages 122–131. Association for Computational Linguistics.

Antoine Chaffin. 2025. Reason-moderncolbert.

Antoine Chaffin and Raphaël Sourty. 2025. Pylate: Flexible training and retrieval for late interaction models. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management, CIKM 2025, Seoul, Republic of Korea, November 10-14, 2025*, pages 6334–6339. ACM.

Haodong Chen, Shuai Wang, Yu Yin, Shengyao Zhuang, Guido Zuccon, and Teerapong Leelanupab. 2026a. Iter: Interaction-aware retrieval for agentic search. *Preprint*, arXiv:2608.27912.

Zijian Chen, Xueguang Ma, Shengyao Zhuang, Jimmy Lin, Akari Asai, and Victor Zhong. 2026b. Agentir: Reasoning-aware retrieval for deep research agents. In *Proceedings of the Conference on Language Modeling*.

Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Sahel Sharifymoghaddam, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Yanxi Li, Haoran Hong, Xinyu Shi, Xuye Liu, Hosna Oyarhoseini, Nandan Thakur, Crystina Zhang, Luyu Gao, and 2 others. 2026c. Browsecomp-plus: A fair and disentangled evaluation benchmark for deep search agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 22349–22370, San Diego, California, United States. Association for Computational Linguistics.

Xin Cheng, Xun Wang, Xingxing Zhang, Tao Ge, Si-Qing Chen, Furu Wei, Huishuai Zhang, and Dongyan Zhao. 2024. xrag: Extreme context compression for retrieval-augmented generation with one token. In *Advances in Neural Information Processing Systems*, volume 37.

Gemma-Team, Sherif El Abd, Vaibhav Aggarwal, Robin Algayres, Alek Andreev, Olivier Bachem, Ian Ballantyne, Cormac Brick, Victor Cărbune, Michelle Casbon, Mayank Chaturvedi, Aditya Chawla, Victor Cotruta, Alice Coucke, Phil Culliton, Robert Dadashi, Lucas Dixon, Mohamed Elhawaty, Utku Evci, and 304 others. 2026. Gemma 4 technical report. *Preprint*, arXiv:2607.02770.

Google. 2025. Gemini Deep Research — your personal research assistant. `https://gemini.google/overview/deep-research/`. [Accessed 24-09-2026].

Shuyu Guo, Shuo Zhang, and Zhaochun Ren. 2025a. Enhancing rag efficiency with adaptive context compression. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 24061–24076, Suzhou, China. Association for Computational Linguistics.

Zirui Guo, Xubin Ren, Lingrui Xu, Jiahao Zhang, and Chao Huang. 2025b. Rag-anything: All-in-one rag framework. *Preprint*, arXiv:2510.12323.

Nikita Gupta, Riju Chatterjee, Lukas Haas, Connie Tao, Andrew Wang, Chang Liu, Hidekazu Oiwa, Elena Gribovskaya, Jan Ackermann, John Blitzer, Sasha Goldshtein, and Dipanjan Das. 2026. Deepsearchqa: Bridging the comprehensiveness gap for deep research agents. *arXiv preprint arXiv:2601.20975*.

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Ö. Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. In *Proceedings of the 3rd Conference on Language Modeling*, Montreal, Canada.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. 2025. Search-o1: Agentic search-enhanced large reasoning models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 5420–5438, Suzhou, China. Association for Computational Linguistics.

Zhuofeng Li, Haoxiang Zhang, Cong Wei, Pan Lu, Ping Nie, Yi Lu, Yuyang Bai, Shangbin Feng, Hangxiao Zhu, Ming Zhong, Yuyu Zhang, Jianwen Xie, Yejin Choi, James Zou, Jiawei Han, Wenhu Chen, Jimmy Lin, Dongfu Jiang, and Yu Zhang. 2026. Beyond semantic similarity: Rethinking retrieval for agentic search via direct corpus interaction. *Preprint*, arXiv:2605.05242.

Xiaoyuan Liu, Tian Liang, Dongyang Ma, Deyu Zhou, Haitao Mi, Pinjia He, and Yan Wang. 2026. The pensieve paradigm: Stateful language models with learned memory management. In *The Fourteenth International Conference on Learning Representations*.

Yuanhua Lv and ChengXiang Zhai. 2011. Lower-bounding term frequency normalization. In *Proceedings of the 20th ACM International Conference on Information and Knowledge Management*, CIKM '11, page 7–16, New York, NY, USA. Association for Computing Machinery.

Xing Han Lù. 2024. Bm25s: Orders of magnitude faster lexical search via eager sparse scoring. *Preprint*, arXiv:2407.03618.

Chuan Meng, Litu Ou, Sean MacAvaney, and Jeff Dalton. 2026. Revisiting text ranking in deep research. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 3006–3016.

OpenAI. 2025. Introducing deep research. https://openai.com/index/introducing-deep-research/. [Accessed 24-09-2026].

OpenAI, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, Che Chang, and 107 others. 2025. gpt-oss-120b & gpt-oss-20b model card. *Preprint*, arXiv:2508.10925.

K. Pernice, K. Whitenton, J. Nielsen, and Nielsen Norman Group. 2014. *How People Read on the Web: The Eyetracking Evidence*. Nielsen Norman Group.

Thinh Pham, Nguyen Nguyen, Pratibha Zunjare, Weiyuan Chen, Yu-Min Tseng, and Tu Vu. 2026. Sealqa: Raising the bar for reasoning in search-augmented language models.

Shreyas Subramanian, Adewale Akinfaderin, Yanyan Zhang, Ishan Singh, Mani Khanuja, Sandeep Singh, and Maira Ladeira Tanke. 2025. Keyword search is all you need: Achieving rag-level performance without vector databases using agentic tool use. *Preprint*, arXiv:2602.23368.

Tongyi DeepResearch Team, Baixuan Li, Bo Zhang, Dingchu Zhang, Fei Huang, Guangyu Li, Guoxin Chen, Huifeng Yin, Jialong Wu, Jingren Zhou, and 1 others. 2025. Tongyi deepresearch technical report. *arXiv preprint arXiv:2510.24701*.

Tu Vu, Mohit Iyyer, Xuezhi Wang, Noah Constant, Jerry Wei, Jason Wei, Chris Tar, Yun-Hsuan Sung, Denny Zhou, Quoc Le, and Thang Luong. 2024. Freshllms: Refreshing large language models with search engine augmentation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 13697–13720, Bangkok, Thailand. Association for Computational Linguistics.

Shuai Wang, Haodong Chen, Yu Yin, Shengyao Zhuang, Bevan Koopman, and Guido Zuccon. 2026. Search, inspect, fetch: Exploiting boolean retrieval for deep-research agents. *Preprint*, arXiv:2608.02751.

Xiaohua Wang, Zhenghua Wang, Xuan Gao, Feiran Zhang, Yixin Wu, Zhibo Xu, Tianyuan Shi, Zhengyuan Wang, Shizheng Li, Qi Qian, Ruicheng Yin, Changze Lv, Xiaoqing Zheng, and Xuanjing Huang. 2024. Searching for best practices in retrieval-augmented generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 17716–17736, Miami, Florida, USA. Association for Computational Linguistics.

Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. 2025. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*.

Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, and Jingren Zhou. 2026. Resum: Unlocking long-horizon search intelligence via context summarization. *Preprint*, arXiv:2509.13313.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.

Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Wenbiao Yin, Zile Qiao, Xinyu Wang, Pengjun Xie, Fei Huang, Jingren Zhou, Siheng Chen, and Yong Jiang. 2026. Agentfold: Long-horizon web agents with proactive context folding. In *The Fourteenth International Conference on Learning Representations*.

Howard Yen, Ashwin Paranjape, Yuhao Zhang, Thejas Venkatesh, Jack Hessel, and Danqi Chen. 2025. Lost in the maze: Overcoming context limitations in long-horizon agentic search. *Preprint*, arXiv:2510.18939.

Alex L. Zhang, Tim Kraska, and Omar Khattab. 2026a. Recursive language models. *Preprint*, arXiv:2512.24601.

Yunfan Zhang, Kathleen McKeown, and Smaranda Muresan. 2026b. Livenewsbench: Evaluating web search agents with freshly curated news. In *Forty-third International Conference on Machine Learning*.

Shengyao Zhuang, Yuansheng Ni, Hengxin Fun, Jimmy Lin, and Xueguang Ma. 2026. Towards retrieving interaction spaces for agentic search. *Preprint*, arXiv:2606.06880.

	Seal-Hard		BrowseComp-Plus	
	Recall	# Search	Recall	# Search
GEMMA-4-E4B				
REACT	42.04	2.02	15.83	3.91
SLIM	49.02	2.68	14.07	3.25
REACT+RR	47.17	2.29	13.34	3.32
GPT-OSS-120B				
REACT	41.74	1.64	48.62	13.25
SLIM	48.29	2.45	40.19	8.14
REACT+RR	48.28	2.06	41.72	10.48

Table 3: **Retrieval recall.** We report retrieval recall—the percentage of reference (gold) URLs retrieved—and the average number of search calls (# Search) for each method on Seal-Hard and BrowseComp-Plus. Although RR substantially reduces cost, it does not markedly change retrieval recall.

A Experiments

A.1 Additional Observations

Results in Table 1 suggest that the effect of RR depends on the backbone model's capacity. For GEMMA-4-E4B, RR improves average accuracy by 3.65 points over REACT, whereas for GPT-OSS-120B the improvement is negligible, with the benefit appearing mainly as cost reduction. This may be attributed to the fact that GPT-OSS-120B is more capable of long-context reasoning than GEMMA-4-E4B, as reflected by the results with a long-established framework REACT across benchmarks.

A.2 Retrieval recall.

Given the effect of RR on cost and accuracy, we further investigate its effect on an important metric of search: retrieval recall. We compare REACT+RR against REACT and SLIM on Seal-Hard and BrowseComp-Plus, which provide reference URLs for each question. Table 3 reports retrieval recall (R)—the proportion of reference URLs found through each method's search process, and the average number of search calls. On Seal-Hard, REACT+RR achieves 47.17% recall (2.29 searches) with GEMMA-4-E4B and 48.28% recall/2.06 searches with GPT-OSS-120B, above REACT (42.04%/2.02 and 41.74%/1.64) and close to SLIM (49.02%/2.68 and 48.29%/2.45). By contrast, on BrowseComp-Plus, the retrieval recall of REACT+RR is lower than that of REACT (13.34% versus 15.83% with GEMMA-4-E4B, 41.72% versus 48.62% with GPT-OSS-120B). Overall, there is no adversarial effect of RR on the retrieval recall.

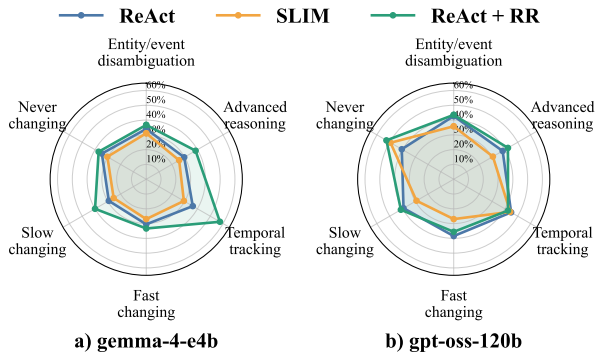


Figure 5: **Performance of REACT variants across question types in Seal-Hard**. Although the gains are not uniform, REACT+RR outperforms REACT and SLIM across a broad range of question types.

A.3 Effect across question types

Because REACT+RR significantly outperforms REACT and SLIM on Seal-Hard, we further analyze their performance across question types and ask whether RR is more beneficial for some types than others. We break down performance according to the Seal-Hard question taxonomy. As shown in Table 4, the Seal-Hard benchmark contains questions from several categories. We note that these categories are not mutually exclusive, and a single question may belong to multiple categories. In our analysis by question type, we omit two categories, `false-premise` and `cross-lingual reasoning`, due to their small sample sizes.

As observed in Figure 5, the performance gain of REACT+RR is broad but not uniform across question types and backbones. For example, on `temporal tracking` questions, REACT+RR improves REACT by 21.2 points with GEMMA-4-E4B and -2.4 points with GPT-OSS-120B; on `advanced reasoning` questions, it improves REACT by 8.9 points with GEMMA-4-E4B and 4.2 points with GPT-OSS-120B. As 72% of Seal-Hard questions are `advanced reasoning`, the overall performance gain reflects the gain in this category.

A.4 Cost Model

We note that 1) deep learning models are orders of magnitude more expensive in computational overhead and energy consumption compared to NLP tools that operate on word level, and 2) in the evaluated frameworks, the backbone LLM is prompted to determine the next action and/or reason concisely (less than 200 output tokens per LLM call on average). For these reasons, the cost of lexical-based filtering (e.g., sparse-search filtering in RR and its counterpart in SLIM) and the cost of output tokens are negligible compared to the cost of input tokens. Thus, the cost computed on the number of input tokens processed by the backbone LLM and the embedding model, plus the cost for web search is a reasonable proxy for the computational cost of each baseline.

In Table 5, we show statistics of each baseline on Seal-Hard with backbone GEMMA-4-E4B: the number of search calls (# Search), the number of input tokens to the backbone LLM (# Tokens_{LLM}) and the embedding model (# $\text{Tokens}_{Embedding}$), and the average cost per thousand samples. The cost (per thousand samples) is computed as follows:

$$\begin{aligned}\textbf{Cost} = &\ \#\,\text{Search} * \text{Cost}_{Search} \\ &+\ \#\,\text{Tokens}_{LLM} * \text{Cost}_{LLM} \\ &+\ \#\,\text{Tokens}_{Embedding} * \text{Cost}_{Embedding}\end{aligned}$$

where Cost_{Search}, Cost_{LLM}, and $\text{Cost}_{Embedding}$ are the costs of 1k search calls, 1k input tokens to the backbone LLM, and 1k input tokens to the embedding model, respectively. In our experiments, we set:

- $\text{Cost}_{Search} = 0.3$: The pricing of Serper API `Ultimate` plan.[4] Although in our experiments, we use key-free DuckDuckGo search, we use the Serper API pricing as a proxy for the cost of web search.
- $\text{Cost}_{LLM} = 0.06 * 10^{-3}$: The pricing of GEMMA-3N-E4B-INSTRUCT in the Together AI provider,[5] which is 0.06 per 1M tokens. We use the pricing of GEMMA-3N-E4B-INSTRUCT as a proxy for the pricing of GEMMA-4-E4B.
- $\text{Cost}_{Embedding} = 2.25 * 10^{-6}$: We use a simple linear extrapolation to estimate the cost of the 149M-parameter embedding model `Reason-ModernColBERT` based on the pricing of GEMMA-3N-E4B-INSTRUCT, as they are dense models. Specifically, the estimated cost of 1k input tokens to `Reason-ModernColBERT` is $0.06 * 10^{-3} * 150/4,000 = 2.25 * 10^{-6}$.

For other benchmarks and backbone models, we apply the same calculation method to compute the cost of each baseline. For GPT-OSS-120B, we set $\text{Cost}_{LLM} = 0.15 * 10^{-3}$, which is its pricing in the Together AI provider. Following this cost model, the Zeroshot baseline has a cost per thousand sam-

[4] https://serper.dev
[5] https://www.together.ai/pricing

Question Type	Count	Elaboration
advanced reasoning	184	Covers multi-hop reasoning, interpreting search-result plots, charts, or tables, and performing counting or calculations
entity/event disambiguation	148	Focuses on distinguishing between similar entities or events
temporal tracking	35	Requires identifying and differentiating instances of entities over time
cross-lingual reasoning	14	The question is in English but answering it requires retrieving and reasoning over non-English sources
false-premise	11	Requires debunking false assumptions
fast-changing	64	Answers typically change within a year
slow-changing	111	Answers change over several years
never-changing	79	Answers never change

Table 4: **Question types in Seal-Hard.**

	# Search	# Tokens$_{LLM}$	# Tokens$_{Embedding}$	Cost
Minimal	1	13,840.16	0	1.13
RLM	2.23	95,005.35	0	6.37
RLM+RR	2.31	63,461.87	0	4.50
REACT	2.02	42,323.18	0	3.15
SLIM	2.68	23,016.4	0	2.18
REACT+RR	2.29	26,745.89	13,150.53	2.32

Table 5: **Cost calculation**. # Search, # Tokens$_{LLM}$, # Tokens$_{Embedding}$, and Cost denote, respectively, the average number of search calls, the average number of input tokens processed by the backbone LLM, the average number of input tokens processed by the embedding model, and the average cost per thousand samples.

ples of approximately 0. That explains why we do not report its cost in Table 1.

A.5 Latency Analysis

Table 6 reports mean end-to-end latency and standard deviation for the core baselines on LiveNews-Bench, Seal-Hard, and DeepSearchQA. We do not report BrowseComp-Plus latency because its task complexity required a multithreaded implementation, and latency measurements for that setup were unavailable at the time of submission. We conduct one-sided Welch two-sample t-tests to check whether RR reduces the latency of the base framework with 95% statistical significance. The results show that RR generally reduces latency relative to its framework baseline, although the effect and the statistical significance vary by benchmark and backbone: REACT+RR is faster on Seal-Hard for both backbones with 95% statistical significance, while on DeepSearchQA the reduction is small and statistically insignificant for GEMMA-4-E4B. We also note that in the reported settings, the latency standard deviations are similar in magnitude to the means, suggesting substantial variance of question complexity.

	LiveNewsBench	Seal-Hard	DeepSearchQA
GEMMA-4-E4B			
RLM	66.21 ± 61.08	70.5 ± 98.22	95.63 ± 67.21
RLM+RR	47.4 ± 40.67	43.63 ± 37.77	87.43 ± 69.77
REACT	78.13 ± 78.32	86.82 ± 110.5	283.42 ± 176.2
SLIM	51.45 ± 47.06	74.94 ± 89.55	205.96 ± 105.76
REACT+RR	65.28 ± 64.27	90.02 ± 112.45	277.52 ± 184.65
GPT-OSS-120B			
RLM	59.29 ± 55.43	67.03 ± 148.58	205.64 ± 247.17
RLM+RR	61.81 ± 48.62	60.39 ± 77.89	145.26 ± 151.14
REACT	39.31 ± 31.01	57.9 ± 67.7	178.22 ± 126.95
SLIM	37.08 ± 50.49	47.81 ± 57.33	144.6 ± 78.84
REACT+RR	32.77 ± 26.22	42.08 ± 44.67	129.73 ± 72

Table 6: **Latency**. The table reports the mean latency (in seconds) and standard deviation for each method and benchmark.

Baseline	LiveNewsBench	Seal-Hard
GEMMA-4-E4B		
REACT	71.5 ($2.52)	31.43 ($3.15)
REACT+RR	74.5 ($1.81)	37.55 ($2.32)
GEMMA-4-26B-A4B (non-thinking)		
REACT	78.0 ($1.64)	47.16 ($3.90)
REACT+RR	78.5 ($1.27)	49.75 ($1.94)
GPT-OSS-20B		
REACT	56.0 ($1.67)	32.93 ($1.58)
REACT+RR	60.5 ($1.20)	32.14 ($0.99)
GPT-OSS-120B		
REACT	68.5 ($3.02)	39.04 ($4.79)
REACT+RR	69.0 ($2.16)	43.43 ($2.76)
KIMI-K2.6 (low reasoning effort)		
REACT	80.5 ($17.91)	61.94 ($29.93)
REACT+RR	81.0 ($17.56)	60.09 ($18.08)

Table 7: Performance and cost of REACT and REACT+RR on LiveNewsBench and Seal-Hard with five backbone models spanning different sizes, model families, and thinking budgets.